REMEDIAL | 20 Q

Remedial Questions

Set 2 Weak Spots

Databricks Certified Data Engineer Associate (exam version May 4, 2026)

- 20 questions targeting the 6 concepts missed on Set 2
- D2 4 (Q6) | D3 6 (Q13, Q20) | D4 4 (Q25) | D7 6 (Q39, Q42)
- Objective line quoted verbatim from the exam outline
- Answer key with explanations, distractor analysis, and doc URLs
- Claims checked against docs.databricks.com (Sep 2026)

Study-only material. Not affiliated with Databricks. Real exam items are not disclosed.

How to Use This Set

These questions target the concepts missed on Set 2 (Q6, Q13, Q20, Q25, Q39, Q42). Each one copies the structure of the official exam guide samples: an **Objective** line quoted from the exam outline, a short scenario with concrete constraints, a one-sentence question, and four options (A to D) with exactly one correct answer. Answers and explanations are collected at the end, as in the exam guide, so you can attempt the questions without seeing them.

Section	Domain	Exam weight	Questions	Range
Section 2	Data Ingestion and Loading	21%	4	Q1-Q4
Section 3	Data Transformation and Modeling	22%	6	Q5-Q10
Section 4	Working with Lakeflow Jobs	16%	4	Q11-Q14
Section 7	Governance and Security	15%	6	Q15-Q20

EXAM FOCUS Pacing: 45 scored questions in 90 minutes is about **2 minutes per question**. Time yourself on this set at the same pace.

TIP Pattern from the official samples: the correct option is usually the **Databricks-native, managed, or automatic** feature. Distractors tend to be (1) workable but sub-optimal manual fixes, (2) legacy or anti-pattern approaches, (3) swapped details such as formats, defaults, or limits, and (4) a real product applied to the wrong use case.

NOTE Product names follow the current exam guide: Lakeflow Jobs, Lakeflow Connect, Lakeflow Spark Declarative Pipelines, and Declarative Automation Bundles (formerly Databricks Asset Bundles).

SECTION 2 Data Ingestion and Loading

Exam weight 21%

Question 1

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

An Auto Loader stream runs on a continuously running job with no trigger specified. The business accepts up to 10 minutes of latency, but the cloud bill shows a very high number of storage API calls from the stream.

Which change reduces the API call volume while keeping the stream running continuously?

- **A.** Delete the checkpoint and restart the stream
- **B.** Add `.trigger(processingTime="10 minutes")` to the write
- **C.** Replace the stream with `.trigger(once=True)` and a job that runs every 10 minutes
- **D.** Increase `cloudFiles.maxFilesPerTrigger`

Question 2

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A scheduled job runs an Auto Loader query with `.trigger(availableNow=True)` and `cloudFiles.maxFilesPerTrigger = 1000`. When the run starts, 50,000 new files are waiting.

How does the run behave?

- **A.** It processes all 50,000 files in one batch and ignores the limit
- **B.** It processes 1,000 files and stops, so the rest wait for the next run
- **C.** It processes all available files in several micro-batches of up to 1,000 files each, then stops
- **D.** It keeps running after the backlog is cleared and waits for new files

Question 3

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A team is migrating a legacy notebook to Databricks Runtime 15.4 LTS. It uses `.trigger(once=True)` to load new files incrementally once per run of a scheduled job.

Which trigger should replace it?

- **A.** Keep `.trigger(once=True)`, which is the recommended option
- **B.** `.trigger(processingTime="0 seconds")`
- **C.** `.trigger(continuous="1 second")`
- **D.** `.trigger(availableNow=True)`

Question 4

Objective: *Enable and detail data ingestion patterns, including batch, streaming, and incremental loading, and import data from sources such as local files, Lakeflow Connect standard connectors, and Lakeflow Connect managed connectors.*

A job runs an Auto Loader `availableNow` query every 4 hours with a checkpoint location set. One run fails after 3 of 5 micro-batches have been committed to the Delta target. The next scheduled run starts normally.

What does the next run process?

- **A.** It resumes from the offsets in the checkpoint and processes only the files that were not committed, without duplicating the committed batches
- **B.** It reprocesses every file from the beginning of the landing folder
- **C.** It skips every file from the failed run, and those files are lost
- **D.** Nothing; the checkpoint must be deleted manually before the stream can run again

SECTION 3 Data Transformation and Modeling

Exam weight 22%

Question 5

Objective: *Implement data cleaning by reading bronze tables with PySpark/SQL, cleaning nulls, standardizing data types, and writing to new silver tables.*

A Databricks SQL query computes `revenue / units`. Some rows have `units = 0`, and the query now fails with a divide-by-zero error. The report should show NULL for those rows.

Which change is most appropriate?

- **A.** Wrap the query in a Python `try/except` block
- **B.** Use `try_divide(revenue, units)`
- **C.** Use `COALESCE(revenue / units, 0)`
- **D.** Increase `spark.sql.shuffle.partitions`

Question 6

Objective: *Implement data cleaning by reading bronze tables with PySpark/SQL, cleaning nulls, standardizing data types, and writing to new silver tables.*

An analyst's query used to return NULL for malformed casts in an old Hive-compatible environment. In Databricks SQL, on an account created in 2024, the same query fails.

Which statement is correct?

- **A.** The `ANSI_MODE` parameter defaults to TRUE for accounts created on or after October 19, 2022, and `SET ANSI_MODE = FALSE` restores Hive-compatible behavior for the session
- **B.** ANSI mode affects only Python DataFrames, not SQL
- **C.** `ANSI_MODE` is fixed and cannot be changed at session level
- **D.** Under ANSI mode, invalid casts return NULL, so the failure must be caused by something else

Question 7

Objective: *Understand the basic tuning parameters (spark.sql.shuffle.partitions, spark.default.parallelism, spark.executor/driver.memory, spark.sql.autoBroadcastJoinThreshold) and re-measure the performance.*

A legacy PySpark job was moved to **serverless** compute. It depends on invalid casts returning NULL, and it now fails. The team needs a temporary fix while they rewrite the code to use `try_cast`.

Which option works on serverless compute?

- **A.** None; ANSI behavior cannot be changed on serverless, so the job must move to classic compute
- **B.** Downgrade the job to Databricks Runtime 13.3
- **C.** `spark.conf.set("spark.sql.ansi.enabled", "false")`, which is one of the properties serverless supports
- **D.** Add a cluster-scoped init script that disables ANSI mode

Question 8

Objective: *Understand the basic tuning parameters (spark.sql.shuffle.partitions, spark.default.parallelism, spark.executor/driver.memory, spark.sql.autoBroadcastJoinThreshold) and re-measure the performance.*

A serverless notebook builds a daily report whose timestamps must be shown in America/New_York time instead of UTC.

Which configuration is supported on serverless compute for this?

- **A.** `spark.driver.extraJavaOptions=-Duser.timezone=America/New_York`
- **B.** An init script that sets the `TZ` environment variable
- **C.** Choose an instance type in the us-east region
- **D.** `spark.conf.set("spark.sql.session.timeZone", "America/New_York")`

Question 9

Objective: *Understand the basic tuning parameters (spark.sql.shuffle.partitions, spark.default.parallelism, spark.executor/driver.memory, spark.sql.autoBroadcastJoinThreshold) and re-measure the performance.*

A job moving to serverless compute used a cluster-scoped init script to install an internal Python package and pin package versions.

How should these dependencies be handled on serverless?

- **A.** Declare them as notebook dependencies or in a base environment
- **B.** Keep the cluster-scoped init script
- **C.** Add the package as Maven coordinates on the compute
- **D.** Use an instance pool built from a custom image

Question 10

Objective: *Understand the basic tuning parameters (spark.sql.shuffle.partitions, spark.default.parallelism, spark.executor/driver.memory, spark.sql.autoBroadcastJoinThreshold) and re-measure the performance.*

A pandas UDF in a nightly job needs several GB of memory per call and fails on serverless compute. The team wants to choose memory-optimized instance types and tune `spark.executor.memory`.

What is the most appropriate compute choice?

- **A.** Stay on serverless and set `spark.executor.memory = 16g`
- **B.** Run the job on classic job compute with memory-optimized instances and the needed Spark memory settings
- **C.** Stay on serverless and only increase `spark.sql.shuffle.partitions`
- **D.** Run the UDF on a serverless SQL warehouse

SECTION 4 Working with Lakeflow Jobs

Exam weight 16%

Question 11

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

After a daily load, the job must notify the data team **only when** the loaded row count falls below 1,000. The check must run as part of the same job.

Which task should be added after the load task?

- **A.** A dashboard task that refreshes the row count dashboard
- **B.** A job-level notification on successful completion
- **C.** A SQL task of type Alert that evaluates a SQL alert on a serverless or pro SQL warehouse
- **D.** A duration warning threshold on the load task

Question 12

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

A dashboard task is added to a job so the published dashboard refreshes after the pipeline finishes. The task is set to use the team's classic SQL warehouse, and it cannot run.

What should the data engineer change?

- **A.** Replace the dashboard task with a notebook task
- **B.** Add retries to the dashboard task
- **C.** Change the task to a SQL alert task
- **D.** Select a serverless or pro SQL warehouse for the dashboard task

---

Question 13

Objective: *Configure common tasks (notebook, SQL query, dashboard, and pipeline tasks) and their dependencies using Lakeflow Jobs and its DAG-based task graph*

A team keeps an `.sql` file with several semicolon-separated statements in their GitHub repository at `etl/silver/build.sql`. They want a job to run that file directly.

Which task configuration should be used?

- **A.** A SQL task of type File with the source set to the Git provider and the relative path `etl/silver/build.sql`
- **B.** A dashboard task that points to the file
- **C.** A SQL task of type Alert
- **D.** A notebook task with the SQL pasted into a `%sql` cell

---

Question 14

Objective: *Implement control flows (retries and conditional tasks such as branching and looping) using Lakeflow Jobs for pipeline orchestration*

A task has 3 retries configured. The on-call engineer wants an email for **every** failed attempt of that task, not just the final job result.

How should the notifications be configured?

- **A.** A job-level notification on failure
- **B.** A task-level notification on failure for that task
- **C.** A dashboard task with the engineer as a subscriber
- **D.** A SQL alert task that checks the job status

SECTION 7 Governance and Security

Exam weight 15%

Question 15

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

Five days after running `ALTER TABLE main.sales.orders SET MANAGED`, the team finds that an external, non-Databricks consumer still needs the original external location. They must revert the conversion.

What should the data engineer do?

- **A.** `DROP TABLE` and recreate the table as external
- **B.** `RESTORE TABLE main.sales.orders TO VERSION AS OF` the version before the conversion
- **C.** `DEEP CLONE` the table back to the external path
- **D.** `ALTER TABLE main.sales.orders UNSET MANAGED`

Question 16

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

After converting an external Delta table with `SET MANAGED`, the streaming jobs that read from and write to the table start failing.

What is the documented fix?

- **A.** Restart the streams with the same configurations, so they automatically use path-based redirect
- **B.** Delete the stream checkpoints and reprocess everything from scratch
- **C.** Run `UNSET MANAGED` because streaming is not supported on managed tables
- **D.** Change the streams to batch jobs

Question 17

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

A data engineer with SELECT and MODIFY on the external Delta table `main.ops.events` tries to run `ALTER TABLE main.ops.events SET MANAGED`, and the command fails with a permission error.

Which permission is required to convert this external table?

- **A.** SELECT on the table
- **B.** Ownership of the table
- **C.** MODIFY on the schema
- **D.** USE CATALOG on the catalog

Question 18

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

A team wants to test a risky transformation against a copy of a large production table. Changes must not affect production, and they want to use as little extra storage as possible.

Which approach fits this requirement?

- **A.** `ALTER TABLE ... SET MANAGED` on the production table
- **B.** `CREATE TABLE test.orders DEEP CLONE prod.orders`
- **C.** `CREATE TABLE test.orders SHALLOW CLONE prod.orders`
- **D.** `CREATE VIEW test.orders AS SELECT * FROM prod.orders`

Question 19

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

A data steward was granted `MANAGE` on the catalog `main`. They can grant and revoke privileges for others, but their own `SELECT` on a table in `main` fails.

Why, and what is the fix?

- **A.** MANAGE applies only to the catalog object itself, so it must be granted on each table
- **B.** The steward must become the catalog owner before reading anything
- **C.** MANAGE is a superset of ALL PRIVILEGES, so the failure must be a network issue
- **D.** MANAGE does not grant data access; the steward can grant themselves SELECT, plus USE CATALOG and USE SCHEMA if missing

---

Question 20

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

User `alex` has `ALL PRIVILEGES` on the table `main.hr.salaries`, but is not its owner and does not hold MANAGE. `alex` tries to `GRANT SELECT` on the table to a colleague, and the command fails.

Who could perform this grant?

- **A.** Another user who also has ALL PRIVILEGES on the table
- **B.** The owner of the schema `main.hr`
- **C.** A user who has MODIFY on the table
- **D.** Any user with USE CATALOG and USE SCHEMA

ANSWERS

Answer Key

Answers and explanations

#	Answer	Section
1	**B**	D2
2	**C**	D2
3	**D**	D2
4	**A**	D2
5	**B**	D3
6	**A**	D3
7	**C**	D3
8	**D**	D3
9	**A**	D3
10	**B**	D3

#	Answer	Section
11	**C**	D4
12	**D**	D4
13	**A**	D4
14	**B**	D4
15	**D**	D7
16	**A**	D7
17	**B**	D7
18	**C**	D7
19	**D**	D7
20	**B**	D7

REVIEW

Explanations

Rationale, distractors, and sources

Question 1 Section 2 | reinforces Set 2 Q6

Answer: B Add `.trigger(processingTime="10 minutes")` to the write

Why: With no trigger set, the stream checks for new data every few milliseconds, which the docs warn can generate a high volume of cloud storage API calls and unexpected charges. A `processingTime` trigger starts a micro-batch only at the set interval, and the query stays running between batches.

Distractors: A: loses progress and does nothing about polling. C: `Trigger.Once` is deprecated. D: changes batch size, not how often the source is polled.

Source:
https://docs.databricks.com/aws/en/structured-streaming/triggers

Question 2 Section 2 | reinforces Set 2 Q6

Answer: C It processes all available files in several micro-batches of up to 1,000 files each, then stops

Why: `availableNow` consumes all records that are available when the run starts, as incremental batches whose size is controlled by rate-limit options such as `maxFilesPerTrigger` and `maxBytesPerTrigger`, and then stops. This differs from the deprecated `Trigger.Once`, which processed everything in one batch.

Distractors: A: describes the old `Trigger.Once` behavior. B: `availableNow` does not stop after the first batch. D: describes a continuously running trigger.

Source:
https://docs.databricks.com/aws/en/structured-streaming/triggers

Question 3 Section 2 | reinforces Set 2 Q6

Answer: D `.trigger(availableNow=True)`

Why: `Trigger.Once` is deprecated in Databricks Runtime 11.3 LTS and above, and the docs direct users to `AvailableNow`. It keeps the same scheduled, incremental, stop-when-done behavior and also respects rate limits.

Distractors: A: deprecated. B: runs continuously. C: Databricks does not support or recommend continuous processing mode and points to real-time mode for low latency.

Source:
https://docs.databricks.com/aws/en/structured-streaming/triggers

Question 4 Section 2 | reinforces Set 2 Q6

Answer: A It resumes from the offsets in the checkpoint and processes only the files that were not committed, without duplicating the committed batches

Why: Checkpoints store the source offsets and the commit records for each micro-batch, so a restarted query resumes exactly where it left off without reprocessing data. This is what gives exactly-once processing. Each query must have its own checkpoint location.

Distractors: B and C contradict how checkpoints track offsets. D: deleting the checkpoint would cause reprocessing.

Source:
https://docs.databricks.com/aws/en/structured-streaming/checkpoints

Question 5 Section 3 | reinforces Set 2 Q13

Answer: B Use `try_divide(revenue, units)`

Why: Under ANSI mode, division by zero raises an error. `try_divide` behaves like the `/` operator except that it returns NULL instead of throwing an exception when dividing by 0.

Distractors: A: catches the failure but returns no result. C: the division still fails before COALESCE is evaluated. D: unrelated to the error.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-ansi-compliance

Question 6 Section 3 | reinforces Set 2 Q13

Answer: A The `ANSI_MODE` parameter defaults to TRUE for accounts created on or after October 19, 2022, and `SET ANSI_MODE = FALSE` restores Hive-compatible behavior for the session

Why: `ANSI_MODE` controls how built-in functions and casts behave, and its system default is TRUE for accounts created on or after October 19, 2022. `SET ANSI_MODE = FALSE` switches the session to Hive-compatible behavior. The cleaner long-term fix is the `try_` functions, such as `try_cast`.

Distractors: B and C: both contradict the docs. D: the opposite is true; ANSI mode raises errors on invalid casts.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/parameters/ansi_mode
https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-ansi-compliance

Question 7 Section 3 | reinforces Set 2 Q13 / Q20

Answer: C `spark.conf.set("spark.sql.ansi.enabled", "false")`, which is one of the properties serverless supports

Why: `spark.sql.ansi.enabled` is on the short list of Spark properties that can be set on serverless notebooks and jobs. Setting it to false is a stopgap; the durable fix is to use `try_` functions.

Distractors: A: the property is supported on serverless. B: serverless does not let you pick a runtime version like this. D: serverless does not support compute-scoped init scripts.

Source:
https://docs.databricks.com/aws/en/spark/conf
https://docs.databricks.com/aws/en/compute/serverless/limitations

Question 8 Section 3 | reinforces Set 2 Q20

Answer: D `spark.conf.set("spark.sql.session.timeZone", "America/New_York")`

Why: `spark.sql.session.timeZone` is one of the Spark properties that serverless notebooks and jobs allow you to set. Most other Spark and JVM settings are not supported on serverless.

Distractors: A: not supported on serverless. B: serverless has no compute-scoped init scripts. C: serverless does not let you select instance types.

Source:
https://docs.databricks.com/aws/en/spark/conf

Question 9 Section 3 | reinforces Set 2 Q20

Answer: A Declare them as notebook dependencies or in a base environment

Why: Serverless does not support compute-scoped init scripts, compute-scoped libraries, Maven coordinates, or instance pools. The docs say to use a base environment or notebook dependencies to install packages and configure Python.

Distractors: B, C, and D are all listed as unsupported on serverless compute.

Source:
https://docs.databricks.com/aws/en/compute/serverless/limitations

Question 10 Section 3 | reinforces Set 2 Q20

Answer: B Run the job on classic job compute with memory-optimized instances and the needed Spark memory settings

Why: Serverless does not let you choose instance types or set most Spark properties, and user code such as UDFs, `map`, and `mapPartitions` is limited to 1 GB of memory there. Workloads that need custom memory sizing belong on classic compute, where instance types and Spark memory settings can be configured.

Distractors: A: memory properties cannot be set on serverless. C: does not raise the per-UDF memory limit. D: SQL warehouses run SQL, not pandas UDF jobs.

Source:
https://docs.databricks.com/aws/en/compute/serverless/limitations
https://docs.databricks.com/aws/en/spark/conf

Question 11 Section 4 | reinforces Set 2 Q25

Answer: C A SQL task of type Alert that evaluates a SQL alert on a serverless or pro SQL warehouse

Why: A SQL task can evaluate a SQL alert, which checks a condition on a query result and notifies subscribers when the condition is met. SQL tasks require a serverless or pro SQL warehouse.

Distractors: A: refreshes the dashboard whether or not the condition is met. B: fires on every success. D: measures runtime, not data.

Source:
https://docs.databricks.com/aws/en/jobs/sql
https://docs.databricks.com/aws/en/sql/user/alerts/

Question 12 Section 4 | reinforces Set 2 Q25

Answer: D Select a serverless or pro SQL warehouse for the dashboard task

Why: Dashboard tasks must use a serverless or pro SQL warehouse. If no warehouse is selected, the task uses the dashboard's default warehouse.

Distractors: A and C: these do not refresh the dashboard. B: retries do not fix an unsupported warehouse type.

Source:
https://docs.databricks.com/aws/en/jobs/dashboard

Question 13 Section 4 | reinforces Set 2 Q25

Answer: A A SQL task of type File with the source set to the Git provider and the relative path `etl/silver/build.sql`

Why: A SQL task of type File runs `.sql` files that contain multiple statements separated by semicolons. The file can come from the workspace, including Git folders, or directly from a Git provider by relative path.

Distractors: B and C: wrong task types. D: duplicates code outside version control.

Source:
https://docs.databricks.com/aws/en/jobs/sql

Question 14 Section 4 | reinforces Set 2 Q25

Answer: B A task-level notification on failure for that task

Why: The docs note that job-level notifications are not sent when failed tasks are retried, and recommend task notifications to get an alert after every failed attempt. Notifications support start, success, failure, duration thresholds, and streaming backlog events, sent to email, Slack, Teams, PagerDuty, or webhooks.

Distractors: A: not sent for attempts that are retried. C and D: misuse dashboard and alert tasks for job monitoring.

Source:
https://docs.databricks.com/aws/en/jobs/notifications

Question 15 Section 7 | reinforces Set 2 Q39

Answer: D `ALTER TABLE main.sales.orders UNSET MANAGED`

Why: Within 14 days, `UNSET MANAGED` points the table metadata back to the original external location and keeps any writes made to the managed copy. Databricks retains the external data for those 14 days specifically so you can roll back.

Distractors: A: loses metadata and grants. B: time travel restores data versions, not the table's storage type. C: creates a separate table with its own history.

Source:
https://docs.databricks.com/aws/en/tables/convert-to-managed

Question 16 Section 7 | reinforces Set 2 Q39

Answer: A Restart the streams with the same configurations, so they automatically use path-based redirect

Why: The docs state that existing read and write streams fail after the conversion, and that restarting them with the same configurations makes them use path-based redirect automatically.

Distractors: B: unnecessary reprocessing. C: managed tables support streaming. D: unnecessary redesign.

Source:
https://docs.databricks.com/aws/en/tables/convert-to-managed

Question 17 Section 7 | reinforces Set 2 Q39

Answer: B Ownership of the table

Why: Converting an external table requires ownership of the table, and the table must be in Delta format. Converting a foreign table instead requires OWNER or MANAGE on the table plus CREATE on the external location.

Distractors: A, C, and D: data access and usage privileges are not enough to convert a table.

Source:
https://docs.databricks.com/aws/en/tables/convert-to-managed

Question 18 Section 7 | reinforces Set 2 Q39

Answer: C `CREATE TABLE test.orders SHALLOW CLONE prod.orders`

Why: A shallow clone copies only the metadata and references the source data files, so it uses less storage and compute. A clone has its own history, and changes to it do not affect the source, which makes shallow clones suited for testing.

Distractors: A: changes the production table itself. B: copies all of the data, which uses the most storage. D: a view cannot be modified independently of the source.

Source:
https://docs.databricks.com/aws/en/delta/clone

Question 19 Section 7 | reinforces Set 2 Q42

Answer: D MANAGE does not grant data access; the steward can grant themselves SELECT, plus USE CATALOG and USE SCHEMA if missing

Why: Users with MANAGE are not automatically given all privileges on the object; each specific privilege must be granted separately, and they can grant those privileges to themselves. MANAGE on a catalog also applies to child schemas and tables.

Distractors: A: MANAGE does cascade to children. B: ownership is not required. C: MANAGE and ALL PRIVILEGES are separate, and MANAGE does not include data access.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

Question 20 Section 7 | reinforces Set 2 Q42

Answer: B The owner of the schema `main.hr`

Why: Privileges can be granted by the object's owner, a user with MANAGE on the object, the owner of the containing catalog or schema, or a metastore admin. ALL PRIVILEGES explicitly does not include MANAGE, which is why `alex` cannot grant.

Distractors: A, C, and D: none of these privileges allow managing grants.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

EXAM FOCUS All answers are grounded in docs.databricks.com, checked Oct 4, 2026. Key takeaways: (1) `availableNow` plus a job schedule gives incremental batch ingestion; (2) ANSI mode is on by default, so use `try_` functions; (3) serverless allows only a short list of Spark properties; (4) Dashboard task refreshes dashboards, Alert evaluates conditions; (5) `SET MANAGED` converts a table in place and `UNSET MANAGED` rolls back within 14 days; (6) ALL PRIVILEGES does not include MANAGE.